

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

15 - Loops

Introduction

- Decisions are important both for
 - Choosing between two alternatives
 - *if* statements
 - Iterating a computation
 - Loops
- Same assembly instructions are building blocks for both cases
 - beq
 - bne

Loops

- Used for Iterating a computation
- Loops are commonly represented in programming languages using
 - `for`
 - `while`
 - `do-while`

Example

- A traditional loop in Java:

```
while (grade[i] == k)  
    i += 1;
```

- Assume that i and k correspond to registers \$s3 and \$s5 and base of array grade is in \$s6. What is MIPS assembly code corresponding to this code segment?

Example (cont.)

- First, load `grade[i]` into temporary register
 - Before we can load `grade[i]`, need to have its address
- Next, multiply index `i` by 4 due to byte addressing problem
 - Use `sll`, shifting left by 2 bits multiplies by 2^2 or 4
 - Add label `Loop`, so can branch back to that instruction at end of loop

```
Loop: sll $t1,$s3,2 # Temp reg $t1 = i * 4
```

Example (cont.)

- To get address of grade[i], add \$t1 and base of grade in \$s6
`add $t1,$t1,$s6 # $t1 = address of grade[i]`
- Now, use that address to load grade[i] into temporary register
`lw $t0,0($t1) # Temp reg $t0 = grade[i]`
- Next instruction performs the loop test, exiting if save[i] ≠ k:
`bne $t0,$s5, Exit # go to Exit if grade[i] ≠ k`

Example (cont.)

- Next instruction adds 1 to i:

```
addi $s3,$s3,1 # i = i + 1
```

- End of loop branches back to the *while* test at top of loop
- Just add Exit label after it

```
j Loop # go to Loop
```

```
Exit:
```

Example (cont.)

- Java code:

```
while (grade[i] == k)
    i += 1;
```

- Compiled MIPS code:

```
Loop:  sll $t1,$s3,2
      add $t1,$t1,$s6
      lw  $t0,0($t1)
      addi $s3, $s3, 1
      bne $t0,$s5, Exit
      j  Loop
```

Exit:

Basic Block

- Sequences of instructions that end in a branch are so fundamental to compiling
- They are given their own buzzword: **basic block**
 - A sequence of instructions
 - without branches, except possibly at end, and
 - without branch targets or branch labels, except possibly at the beginning
- One of the first early phases of compilation is breaking the program into basic blocks